

BTO3102 TEAM 5 NETQUES MOBILE APPLICATION DEVELOPMENT REPORT

Computer Networks and Operating Systems Educational Game

- **Project Name:** NetQues
- **Project Type:** Mobile educational game prototype
- **Technology Stack:** Flutter, Flame, Dart, SQLite, SharedPreferences
- **Repository:** <https://github.com/NurullahDAGHAN/OS-Project-xMeine.git>
- **Game Demo Video:**
https://drive.google.com/file/d/1_jEi3nQnuKhrGf7IGd_gNHZ98vg5Wx4e/view?usp=drive_link
- **Team Information:** TEAM-5
- **Team Members:** Emre Çoban, Muazzez Şen, Esranur Aydın, Nurullah Dağhan, İdris Baki Uzun
- **Project Area:** Computer Networks and Operating Systems
- **Target Audience:** Middle school and early high school students who are beginning to learn basic computing concepts.

1. Project Overview & Academic Background

NetQues is an interactive mobile learning application developed as a joint educational project for Computer Networks and Operating Systems topics. The primary purpose of this application is to teach abstract, complex, and "invisible" technical computing concepts through concrete, game-based tasks, immediate reinforcement, and profile-based gamification rather than presenting them solely as text-heavy definitions.

The core pedagogical framework of NetQues is structurally built on three fundamental pillars of learning theory:

- **Constructivist Learning Approach:** Instead of passively memorizing definitions, students actively construct technical knowledge in their minds by interacting with physical components (e.g., dragging cables, selecting cards) and processing adaptive hint loops.
- **Vygotsky's Socio-Cultural Theory & Zone of Proximal Development (ZPD):** The game introduces concepts through story-driven level mechanics inspired by socio-cultural development. By utilizing flexible difficulty layers, the system provides dynamic "Scaffolding" (İskele Kurma) that adapts to the learner's initial readiness and cognitive pace.
- **Skinner's Operant Conditioning Principle:** The core gameplay relies on immediate positive and negative reinforcement loops. Completing tasks flawlessly rewards the student with 3 stars (positive reinforcer), whereas making errors systematically reduces the star count. This design encourages students to autonomously replay levels to achieve total mastery learning.

2. Design Methodology: The ADDIE Model

The instructional design of NetQues strictly follows the structured phases of the **ADDIE** framework:

2.1. Analysis

- **Content Analysis:** Network and operating system topics were systematically broken down into short, digestible micro-learning objectives where each level targets exactly one concept and one clear task.
- **Target Audience Analysis:** The application explicitly accommodates beginner learners who easily experience cognitive overload or lose motivation when faced with abstract technical literature via pure text.
- **Learning Need:** Crucial system mechanics like Gateways, DNS, Processes, and RAM are inherently invisible to students. NetQues maps these ideas onto concrete visual metaphors and task-based interactions.

2.2. Design

- **Pedagogical Design:** The structural learning flow uses small-step instruction combined with constructivist feedback loops. Making a mistake is treated as a vital learning opportunity; selecting an incorrect card triggers an intelligent hint explaining *why* it is wrong, guiding the user to self-correct.
- **Gamification Design:** Extrinsic and intrinsic motivational drivers are balanced using daily streaks, total stars, locked badges, task checklists, and final achievement certificates.
- **Interface & Universal Design (UDL):** The system features a mobile-friendly layout supporting portrait and landscape orientations. Going beyond basic UI layouts, it embeds Universal Design for Learning principles inside an advanced Settings Menu, allowing students with diverse physical or cognitive needs (e.g., dyslexia, visual fatigue, or attentional variations) to learn comfortably at their own pace.

2.3. Development

- **Architecture:** **Flutter** provides the overarching application management and state shell, while the **Flame** game engine drives the interactive canvas and gameplay scenes.
- **Persistence:** Local progress and secure data are stored using **SQLite**, while user preferences and light session configurations are handled via **SharedPreferences**.
- **Audio & Speech:** Sound effects are managed with **audioplayer**, and synchronized screen-reading is engineered using **flutter_tts**.
- **Localization:** Content is localized seamlessly through an app-level localization layer supporting both Turkish and English.

2.4. Implementation

The developed implementation is a fully operational mobile prototype consisting of user authentication, module navigation, persistent level tracking, audio/speech adjustments, and two comprehensively designed curriculum modules. Students unlock subsequent stages sequentially, earn star rewards, and can freely replay completed nodes to maximize their scores.

2.5.Evaluation

In-App Data Analytics & Formative Assessment: To natively measure instructional effectiveness, evaluation tools are integrated into the database level. The system tracks completion logs, total attempts, hint usage, daily streak metadata, and badge milestones.

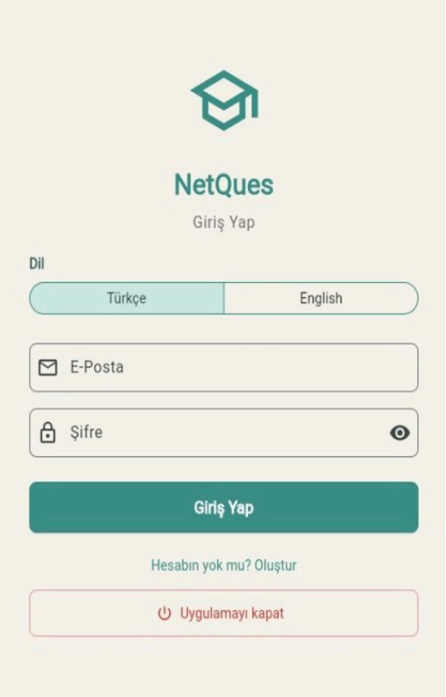
Formative Assessment Feedback: At the end of operational modules, enabling a self-contained evaluation layer that measures the application's true impact on the learner's technical retention.

3. System Architecture and User Experience Flow

3.1.Authentication and Account Management

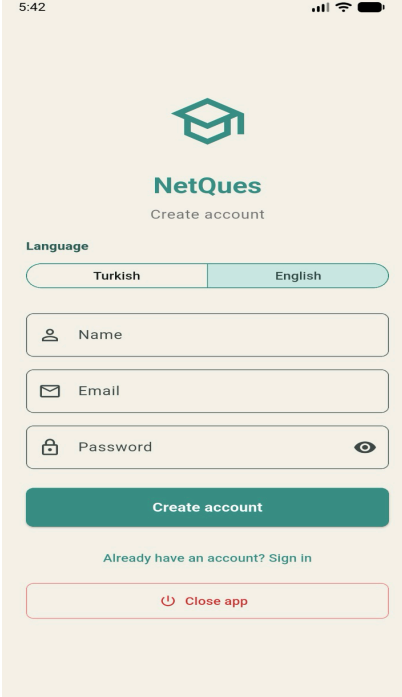
Users register and log in via email and password keys. For local data protection, passwords are encrypted and stored as **SHA-256 hashes** inside the SQLite file. SharedPreferences preserves persistent session data including active login state, user email, and display names.

Login Screen



The Login Screen features the NetQues logo at the top, followed by the text "Giriş Yap". Below this is a language selection bar with "Türkçe" and "English" options. The main form contains three input fields: "E-Posta" (Email), "Şifre" (Password) with a toggle icon, and a "Giriş Yap" button. At the bottom, there is a link "Hesabın yok mu? Oluştur" and a "Uygulamayı kapat" button.

Sign in Screen

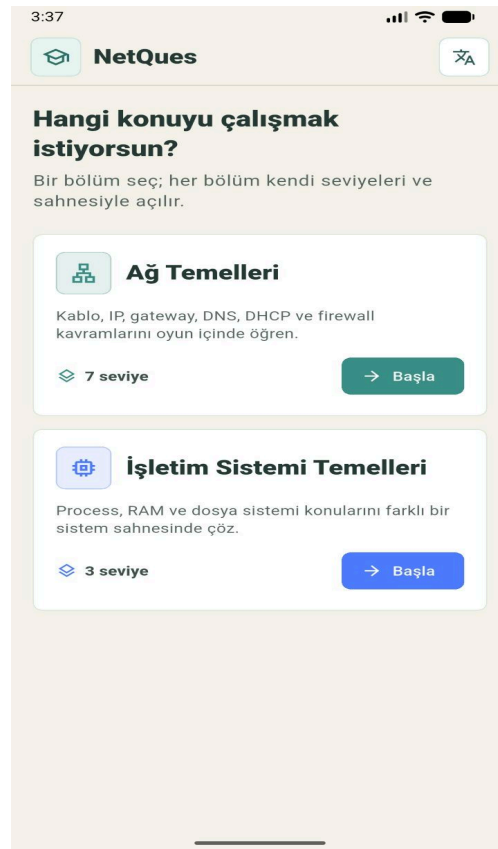


The Sign in Screen features the NetQues logo at the top, followed by the text "Create account". Below this is a language selection bar with "Turkish" and "English" options. The main form contains three input fields: "Name", "Email", and "Password" with a toggle icon, and a "Create account" button. At the bottom, there is a link "Already have an account? Sign in" and a "Close app" button.

3.2.Main Navigation and Core Modules

Upon successful authentication, users face a clean module selection hub containing two primary branches:

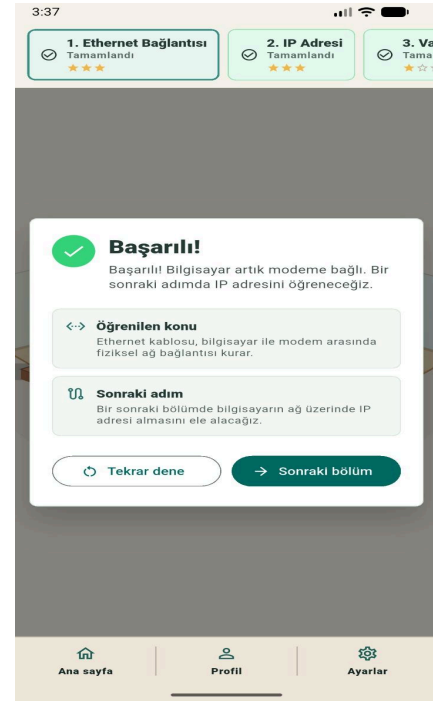
- **Network Basics:** Covers fundamental network architecture layers (Ethernet, IP, Gateway, DNS, Subnet, DHCP, Firewall).
- **Operating System Basics:** Runs an isolated "Operating System Lab" scene focusing on core computing infrastructure (Processes, RAM, File Systems).



3.3.Core Gameplay Loop

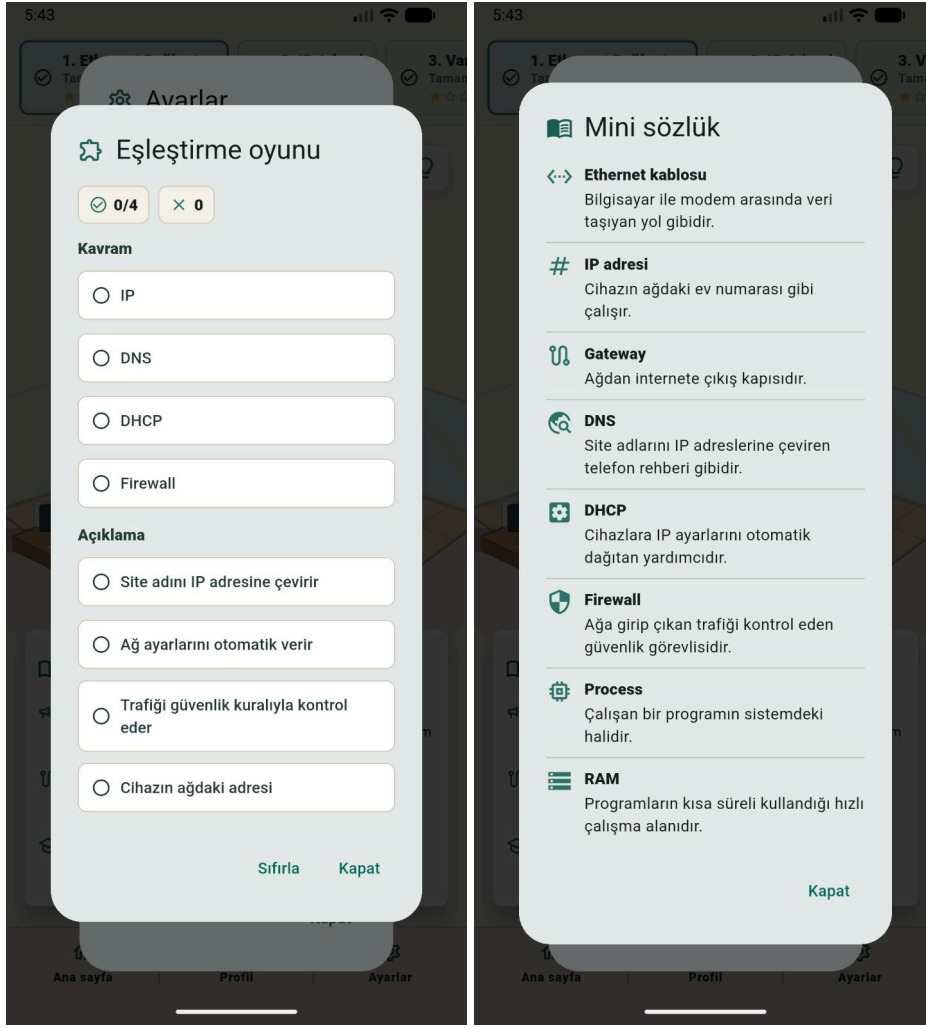
1. **Instruction:** The student reads the contextualized, story-based task instruction.
2. **Action:** The student executes the task by physically drawing a connection cable or choosing an answer card.

3. **Constructive Correction:** If the action is incorrect, the system prevents failure-frustration by delivering a structural hint.
4. **Mastery Confirmation:** If correct, a customized success panel triggers an educational summary note to solidify the conceptual schema.
5. **Progression:** The subsequent level node unlocks automatically.
6. **Replayability:** The student can actively return to completed levels to optimize their star configuration.



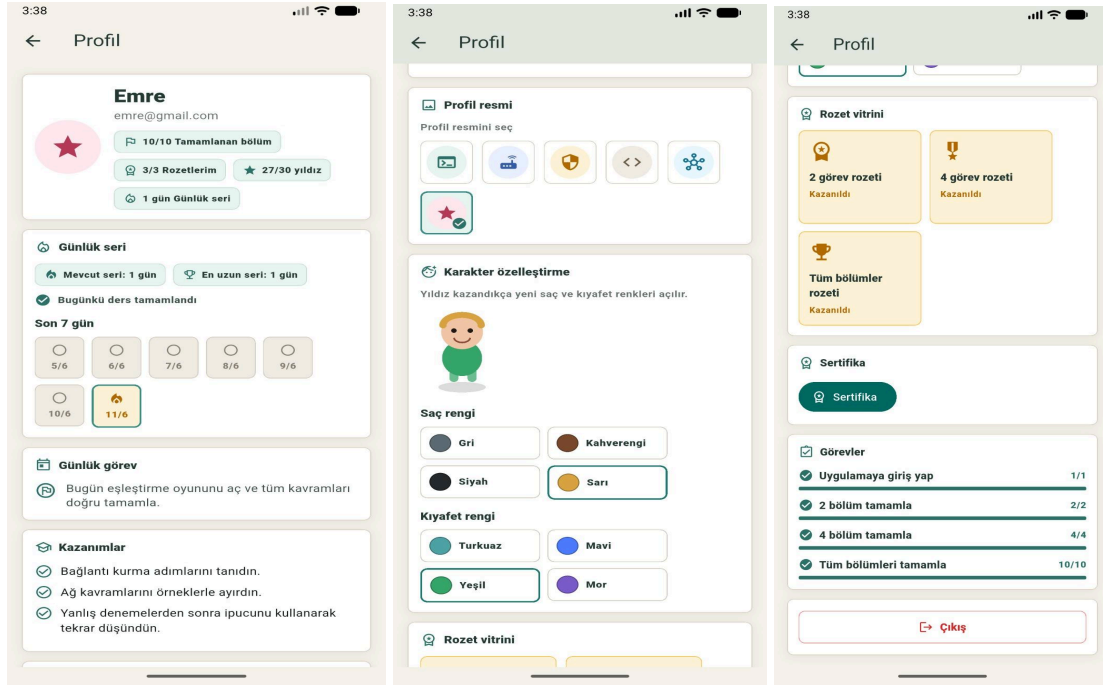
3.4. Game Zone and Reinforcement

To enhance learning retention, the core gameplay is supported by a **Matching Game**. In this activity, students reinforce their knowledge by matching technical terms with their correct definitions. The previously separate matching cards screen was removed from the core user interface to maintain user focus and simplify the overall design. In addition, the application includes a **"Mini Dictionary"**, which helps students quickly review and recall concepts whenever needed.



3.5.Profile Hub & Intrinsic Rewards

The Profile Page visualizes the student's learning journey , showing metrics like completed levels, cumulative stars, active streaks, badges, tasks, and certificates. To convert academic achievement into an **intrinsic motivator**, students spend their collected star rewards to unlock physical avatar character customizations (altering hair and outfit colors).



4. Current Learning Modules & Curricular Mapping

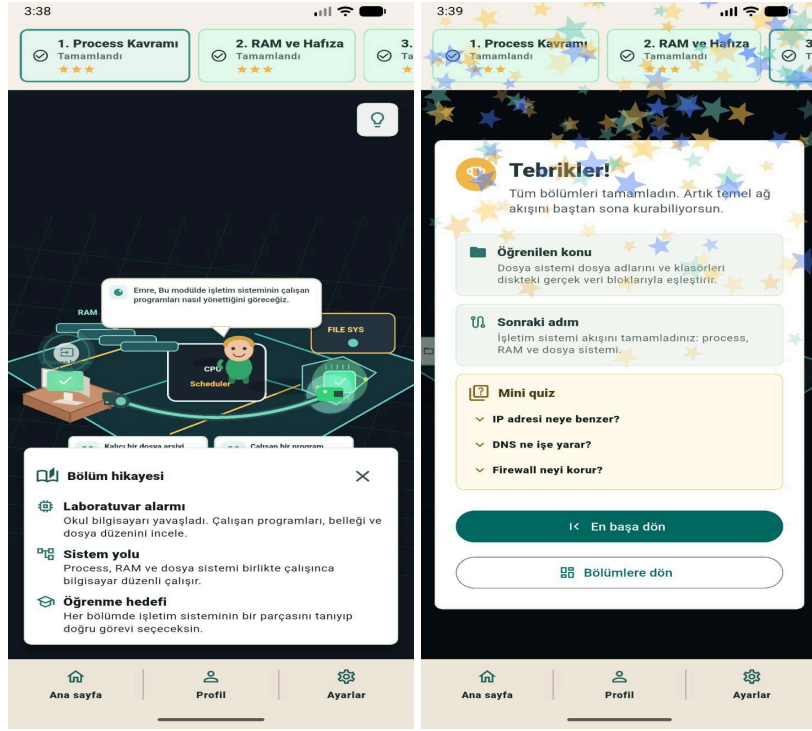
4.1. Network Basics Module

1. **Ethernet Connection:** Drag-and-drop mechanics to physically connect a computer node to a modem asset.
2. **IP Address:** Selection tasks to identify unique device destination markers on a subnet.
3. **Default Gateway:** Determining the correct external routing boundary to exit to the broader internet.
4. **DNS Server:** Simulating the service that resolves human-readable domain names into network IP addresses.
5. **Subnet Mask:** Practical exercises mapping out local broadcast boundaries.
6. **DHCP Service:** Interacting with automated network configuration assignment routines.
7. **Firewall:** Choosing security rulesets to inspect, allow, or drop traffic blocks.



4.2. Operating System Basics Module

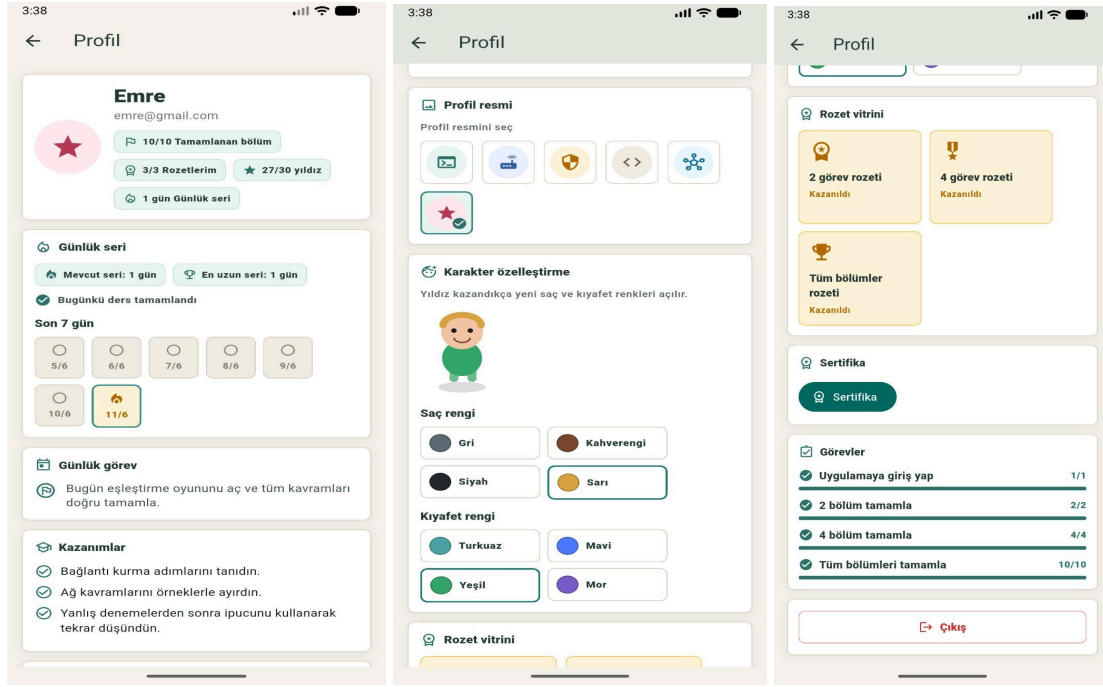
1. **Process:** Visualizing and identifying live, scheduled instances of running application packages.
2. **RAM:** Interacting with temporary working memory tables to monitor active task allocations.
3. **File System:** Managing hierarchical structures organizing persistent user files and disk directories.



5. Inclusive Education & Advanced Settings Layer

While initial design iterations experimented with separate static layout pages (such as Easy Reading, High Contrast, and Color-Blind modes), these specific screens were refactored. Instead, NetQues consolidates accessibility directly into a **Universal Settings Layer** to avoid isolating specialized users:

- **Adaptive Scaffolding Controls:** Guided by Vygotsky's ZPD, users can alter difficulty on the fly. Lowering difficulty increases the depth, clarity, and frequency of instructional hints, establishing an explicit scaffolding system for struggling learners.
- **Advanced Text-to-Speech (TTS) Engine:** To aid auditory learners, visually fatigued students, or children dealing with reading barriers like dyslexia, the system integrates full-text reading synced to the active app language. Sliders for speech-rate and vocal pitch enable students to tune audio output to match their personal cognitive processing speed.
- **Custom Reward Mechanics:** Operant conditioning rewards are adjustable. Students can alternate between high-stimulation particle animations (e.g., confetis) or calmer victory indicators depending on their sensory needs.
- **Sensory Overload Protection:** To support neurodiverse students (such as those on the Autism spectrum or with ADHD), background audio and intensive victory animations can be globally muted or disabled to maintain an emotionally regulated, focused learning atmosphere.



6. Technical and Software Infrastructure

6.1. Main Code Structure

Path / File	Description & Subsystem Responsibility PDF
<code>lib/main.dart</code>	Application bootstrapper, shell, authentication routers, and primary game container switches.
<code>lib/data/</code>	Holds clean authentication models, user progress tracking, streak states, and profile repositories.
<code>lib/game/network_game.dart</code> <code>lib/game/levels/</code>	Handles core Flame canvas instantiation, loop logic, and static technical data vectors for Network/OS stages.
<code>lib/game/components/</code>	Custom UI components, drag targets, and victory message panels drawn directly via code.

<code>lib/l10n/app_localizations.dart</code>	Houses translation lookups for bilingual Turkish/English educational strings.
<code>lib/ui/</code>	Declarative widget files for Login, Module Selection, Level Selection, Profile, and Settings.
<code>assets/audio/</code>	Directory containing uncompressed audio assets for interface and game mechanics feedback.
<code>assets/images/</code>	Directory for vector graphics and local illustration textures.
<code>test/</code>	Automated verification suite checking models, repositories, and UI controllers.

6.2.Data Storage & Persistence Architecture

- **network_cable_demo.db (SQLite):** Dedicated internal database file storing user credentials and secure password hashes.
- **network_training.db (SQLite):** Manages historical gameplay progress, unlock states, completion records, total failures, hint usage, and maximum star records.
- **SharedPreferences Layer:** Acts as a lightweight persistence system cache storing current login state, active username, chosen avatar configurations, current daily streak integer, and audio preferences.
- *Web Platform Adaptability:* When targeted for web browsers, the database repository seamlessly switches to an ephemeral, in-memory system model since native local SQLite operations are restricted in standard browser runtimes.

6.3.Project Dependencies

dependencies: [cite_start]flutter: SDK [cite: 146] [cite_start]flame: ^1.x.x # 2D Game engine canvas layer [cite: 146] [cite_start]sqlite: # Local relational data persistence [cite: 146] [cite_start]shared_preferences: # Light preference key-value storage [cite: 146] [cite_start]crypto: # Encryption library handling SHA-256 data hashing [cite: 146] [cite_start]audioplayers: # Game sound effect triggers [cite: 147] [cite_start]flutter_tts: # Text-To-Speech accessibility voice output [cite: 147] dev_dependencies: [cite_start]flutter_test: SDK [cite: 147] [cite_start]sqlite_common_ffi: # Database mocking tool for automated local testing [cite: 147]

7. Execution, Validation, & Build Commands

To verify, run, and compile the NetQues development workspace locally, use the following operational commands:

Bash

None

```
# Environment Verification
flutter doctor [cite: 149]

# Package Resolution
flutter pub get [cite: 150]

# Local Development Execution
flutter run [cite: 151]

# Deploying and Running via Web Server Protocol
flutter run -d web-server --web-port 8080 [cite: 153]

# Static Code Analysis and Quality Assurance
flutter analyze [cite: 155]

# Executing the Automated Unit/Repository Test Suite
flutter test [cite: 156]

# Compiling Production-Ready Web Build Assets
flutter build web [cite: 160]
```

8. Current System Limitations

- **Authentication Scope:** The current login mechanism relies entirely on a localized sandbox database. Scaling for public distribution would require migrating to an external backend provider (e.g., Firebase Auth or an enterprise API) with email validation and secure password recovery mechanisms.
- **Web Persistence:** The web compilation profile currently relies on non-persistent, in-memory state tracking. True production web deployments would require mapping repositories onto IndexedDB or an external cloud save framework.
- **Visual Presentation Assets:** To maintain project flexibility, most Flame assets and game elements are rendered programmatically via raw engine code lines. Integrating custom professional design illustrations would improve visual engagement in final releases.

9. Conclusion

NetQues stands as a comprehensive, pedagogically sound educational game prototype that successfully demonstrates how dense, abstract computer science curricula can be re-engineered into highly engaging, game-based micro-learning modules. By merging

structured scaffolding, interactive simulations, and inclusive design principles, the application proves that technical computing domains can become highly intuitive, encouraging equal learning opportunities for all aspiring tech students.